

Application of The Ascon-Aead128 Algorithm for IoT Data Security

Ngo Thi Minh Huong*

University of Technology and Education,
The University of Danang, Vietnam

*Contact: ntmhuong@ute.udn.vn

Tran Van Nam*

University of Technology and Education,
The University of Danang, Vietnam

*Contact: trannam6362@gmail.com

Pham Van Phat

University of Technology and Education,
The University of Danang, Vietnam

pvphat@ute.udn.vn

Abstract— Ensuring data security and integrity in resource-constrained embedded systems remains a significant challenge for conventional cryptographic algorithms due to their high computational and energy costs. Lightweight cryptography (LWC) addresses this issue by optimizing performance for constrained hardware platforms. Among LWC candidates, Ascon-AEAD128, selected as a standard by NIST [1], stands out due to its authenticated encryption with associated data (AEAD) scheme, simple permutation-based structure, and high efficiency in both hardware and software implementations. This paper presents the implementation and performance evaluation of Ascon-AEAD128 on two heterogeneous hardware platforms, namely ESP32-C6 and Raspberry Pi 4, representing different classes of embedded devices. Performance is analyzed in terms of throughput (MB/s) and computational cost (cycles/byte) across multiple payload sizes. Experimental results show that, for a 16-byte payload, the throughput reaches 2.449 MB/s (612.48 cycles/byte) on Raspberry Pi 4 and 0.504 MB/s (317.23 cycles/byte) on ESP32-C6. When the payload increases to 128 bytes, the performance improves to 8.834 MB/s (170.41 cycles/byte) and 1.541 MB/s (103.86 cycles/byte), respectively. In addition, an end-to-end LoRa transmission test using an 11-byte weather data payload was conducted. A total of 10,000 encrypted packets were transmitted from the ESP32-C6 node to the Raspberry Pi 4 gateway, with 0 authentication failures and 0 payload recovery errors observed. These results demonstrate that Ascon-AEAD128 is a suitable option for secure data transmission under the tested LoRa-based IoT conditions in resource-constrained embedded systems.

Keywords—Ascon-AEAD128, lightweight cryptography, IoT security, authenticated encryption with associated data, ESP32-C6, Raspberry Pi 4.

I. INTRODUCTION

Conventional cryptographic algorithms such as AES-GCM and ChaCha20-Poly1305 have been widely adopted due to their strong security guarantees and standardization. However, their deployment on resource-constrained embedded systems remains challenging. These algorithms typically require substantial computational resources and often rely on complex operations or hardware-specific optimizations, resulting in high energy consumption and inconsistent performance on low-power microcontrollers.

In this context, Ascon-AEAD128 offers an alternative approach based on a permutation-oriented design. It employs simple bitwise operations, including XOR, AND, and bit rotations, which significantly reduce computational complexity. This structure enables efficient implementation across both 32-bit and 64-bit architectures without requiring platform-specific optimizations. Furthermore, Ascon integrates authenticated encryption with associated data (AEAD), ensuring both confidentiality and data integrity within a unified process, thereby reducing the overhead compared to combining separate cryptographic primitives.

Another key advantage of Ascon lies in its ability to maintain stable performance when processing small payloads, which are common in embedded systems. While many conventional algorithms suffer from initialization and state-processing overhead, Ascon is specifically optimized for short data scenarios, leading to improved overall efficiency in practical deployments.

Therefore, the selection of Ascon-AEAD128 in this work is motivated not only by its standardization but also by its suitability for embedded systems, including low computational cost, simple structure, and comprehensive security guarantees under constrained resources.

II. BACKGROUND AND RELATED WORK

This section presents the theoretical background of data security in embedded systems and provides an overview of the Ascon-AEAD128 algorithm. First, the security requirements and characteristics of the deployment environment are analyzed to clarify the criteria for selecting an appropriate cryptographic scheme. Then, the structure and operating principles of Ascon-AEAD128 are described as the foundation for the implementation and evaluation presented in the following sections.

A. Encryption in IoT Systems

In embedded and distributed IoT systems, data is often transmitted over untrusted channels, increasing the risk of eavesdropping, tampering, and forgery during transmission. Therefore, ensuring confidentiality, integrity, and authentication is a fundamental requirement in secure system design.

To address these requirements, modern cryptographic schemes typically adopt authenticated encryption (AE), which integrates data confidentiality and integrity verification into a unified process. In resource-constrained embedded environments, the selected algorithm must satisfy key criteria, including low computational cost, small memory footprint, and stable performance across heterogeneous hardware platforms.

Therefore, the choice of a cryptographic solution depends not only on its theoretical security but also on its practical deployability, particularly in environments with diverse resource constraints and architectural differences.

B. Overview of the Ascon-AEAD128 Algorithm

1) General Specification

Ascon-AEAD128 is the authenticated-encryption algorithm specified in the NIST Ascon-based lightweight cryptography standard for constrained devices [1]. It provides authenticated encryption with associated data (AEAD), ensuring both confidentiality of the plaintext and integrity of the associated, unencrypted data.

The architecture of Ascon-AEAD128 is based on a permutation-oriented design inspired by the sponge construction, enabling efficient reuse of the internal state during both encryption and decryption processes.

2) Internal State and Parameters

The core operation of Ascon-AEAD128 is based on an internal state of size $S = 320$ bits [1], [2], [5]. This state is divided into five 64-bit words (x_0, x_1, x_2, x_3, x_4). The use of 64-bit words enables efficient utilization of register-based architectures commonly found in modern embedded systems, including RISC-V, Xtensa, and ARM Cortex platforms.

The main parameters of Ascon-AEAD128 are defined as follows:

- Key size (k) and nonce size (N): 128-bits.
- Rate (r): 128-bits.
- Capacity (c): 192-bits, where $S = r + c$.
- Number of initialization/finalization rounds (a): 12
- Number of intermediate permutation rounds (b): 8

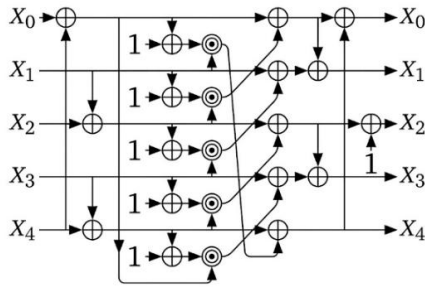


Fig. 1. Ascon S-box.

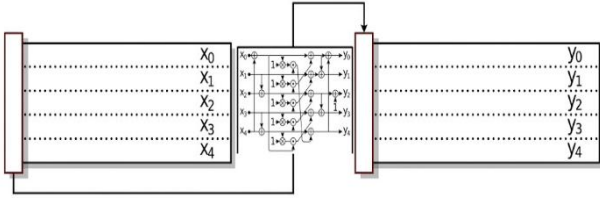


Fig. 2. The Ascon S-box applied on the first column of the state.

3) AEAD Operation of Ascon-AEAD128

Following the NIST Ascon-AEAD128 parameterization, the data processing follows four sequential phases [1], [2], [5]:

Initialization: The 320-bit internal state is initialized using the initialization vector (IV), the secret key K , and the nonce N . The permutation Ascon-p[12] is applied to thoroughly mix the state. Afterward, the key is XORed into the state to strengthen the initialization process.

Associated Data Processing: If associated data is present, it is parsed into 128-bit rate blocks according to the Ascon-AEAD128 rate. Each block is XORed into the rate portion of the state and followed by the intermediate permutation Ascon-p[8]. This process authenticates the associated data without encrypting it.

Plaintext Processing (Encryption): The plaintext is also parsed into 128-bit blocks. For each block, the plaintext is XORed with the rate part of the state to produce the corresponding ciphertext block. The state is then updated using Ascon-p[8]. This structure enables a streaming-like encryption process without requiring full data buffering.

Finalization and Tag Generation: After all data blocks are processed, the secret key is reintroduced into the internal state.

The final permutation Ascon-p[12] is applied to ensure diffusion, and a 128-bit authentication tag is generated. This terminology is used consistently to distinguish the standardized Ascon-AEAD128 algorithm from earlier ASCON-128 and ASCON-128a designs.

4) Permutation Architecture

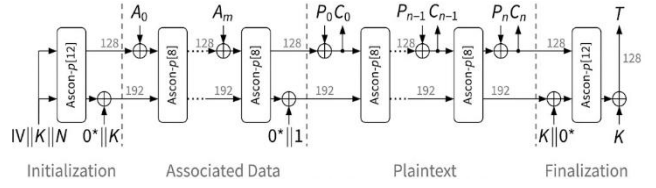


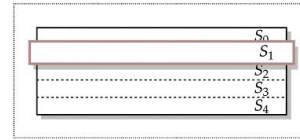
Fig. 3. Structural diagram of the Ascon-AEAD128 authenticated encryption algorithm.

The nonlinearity and diffusion properties of Ascon-AEAD128 rely on its core permutation function. Each permutation round p consists of three transformation layers:

Addition of Round Constants: An 8-bit round constant is XORed into the state word x_2 to break symmetry between rounds and prevent slide attacks.

Nonlinear Substitution Layer: A 5-bit S-box is applied in parallel across 64 slices of the state. The S-box is specifically designed to optimize basic logical operations, including AND, NOT, and XOR, while providing strong resistance against differential and linear cryptanalysis.

Linear Diffusion Layer: This layer provides intra-word diffusion within each 64-bit state word using bitwise rotations and XOR operations. The transformation ensures effective mixing of bits across the state and is defined by the linear equations shown in Fig. 4.



$$\begin{aligned} S_0 &:= S_0 \oplus (S_0 \ggg 19) \oplus (S_0 \ggg 28) \\ S_1 &:= S_1 \oplus (S_1 \ggg 61) \oplus (S_1 \ggg 39) \\ S_2 &:= S_2 \oplus (S_2 \ggg 1) \oplus (S_2 \ggg 6) \\ S_3 &:= S_3 \oplus (S_3 \ggg 10) \oplus (S_3 \ggg 17) \\ S_4 &:= S_4 \oplus (S_4 \ggg 7) \oplus (S_4 \ggg 41) \end{aligned}$$

Fig. 4. Ascon linear layer

III. PROPOSED SYSTEM

A. Overall System Architecture:

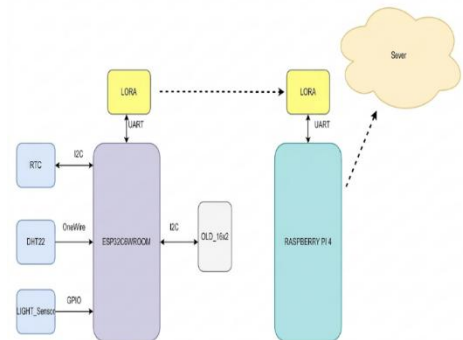


Fig. 5. System architecture

The system is designed based on a distributed IoT architecture, consisting of four main stages: data acquisition, processing, encryption, and transmission. The data flow is implemented through two primary physical components: an edge node (ESP32-C6) [7] and a central gateway (Raspberry Pi 4) [8], which communicate via a LoRa network. The edge node is responsible for environmental data collection, secure encryption, and data transmission. The gateway handles data reception, decryption, authentication, and forwarding to the application layer.

B. Processing and Packetization at the Edge Node (ESP32-C6)

The processing at the edge node begins with acquiring data from sensors. The collected data is organized into a plaintext frame, which includes fields such as timestamp, temperature, humidity, and light intensity. To evaluate the performance of the proposed algorithm, the payload size is configurable, with typical values such as 16 bytes and 128 bytes. The plaintext frame is then processed by the encryption module using the Ascon-AEAD128 algorithm. The microcontroller utilizes a pre-shared secret key K (128-bit) and generates a unique nonce N (128-bit) for each encryption session. The encryption process produces the ciphertext along with a 128-bit authentication tag.

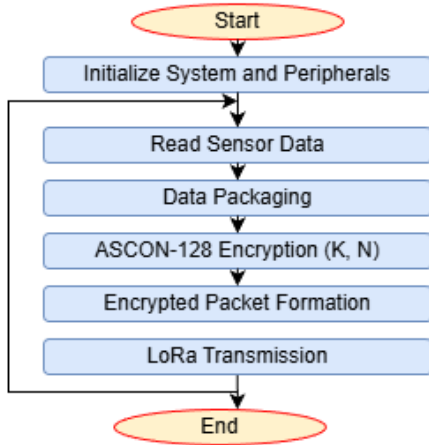


Fig. 6. Operational flowchart of the transmitter node for sensor data acquisition and Ascon-AEAD128 encryption.

The ciphertext and authentication tag are concatenated into a single output frame, eliminating the need for a separate transmission channel for the tag and thereby improving bandwidth efficiency.

C. Authentication and Decryption at the Gateway (Raspberry Pi 4)

At the receiver side, the gateway receives data packets forwarded from the LoRa module and performs frame parsing. The nonce N is extracted directly from the packet header and combined with the locally stored secret key K as input for the decryption process of Ascon-AEAD128. Simultaneously, data integrity is verified by comparing the received authentication tag with the internally computed tag. For valid packets, the plaintext is successfully recovered and forwarded to the application layer for further processing. If the two values do not match, the packet is considered corrupted or tampered with during transmission and is immediately discarded at the network layer.

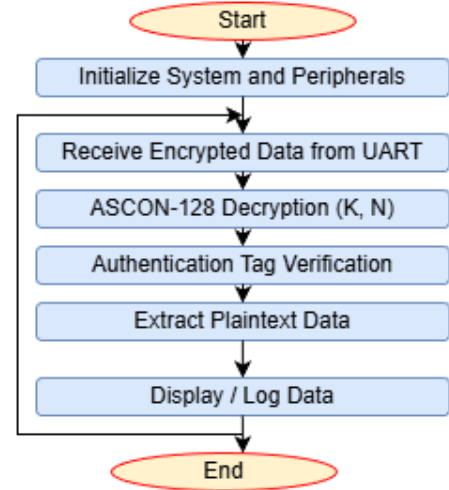


Fig. 7. Operational flowchart of the receiver node for Ascon-AEAD128 decryption and data recovery.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

This section presents the experimental evaluation of the proposed system, including computational performance of Ascon-AEAD128, data accuracy validation, and system-level verification on both ESP32-C6 and Raspberry Pi platforms.

A. Experimental Setup and Measurement Methodology

The experimental system consists of an ESP32-C6 edge node, a Raspberry Pi 4 gateway, and a LoRa communication link. The ESP32-C6 node performs weather data acquisition, plaintext frame construction, Ascon-AEAD128 encryption, and packet transmission. The Raspberry Pi 4 gateway receives the encrypted packets, extracts the nonce and ciphertext, verifies the authentication tag, performs decryption, and recovers the original plaintext payload.

Two types of experiments were conducted. First, a computational benchmark was performed to evaluate the encryption and decryption performance of Ascon-AEAD128 using different payload sizes, including 12, 16, and 128 bytes. The performance was measured in terms of throughput and cycles per byte. Second, an end-to-end LoRa transmission test was conducted to evaluate packet-level integrity and payload recovery under real communication conditions.

For the packet-level reliability test, each transmitted frame consisted of a packet header, a 128-bit nonce, an 11-byte encrypted weather data payload, and a 128-bit authentication tag. A total of 10,000 packets were transmitted from the ESP32-C6 node to the Raspberry Pi 4 gateway. A packet was considered successful only if it was received, successfully authenticated by the Ascon-AEAD128 tag verification process, and correctly recovered to the original 11-byte weather payload.

B. Benchmarking of Ascon-AEAD128

As shown in Table I, the computational performance of Ascon-AEAD128 exhibits a clear dependency on the payload size. Specifically, on the ESP32-C6 platform, the computational cost decreases significantly from 334.22 cycles/byte for a 12-byte payload to an optimized value of 103.86 cycles/byte for a 128-byte payload. This trend highlights the impact of initialization overhead introduced by the internal state setup phase of the algorithm. However, this overhead is effectively amortized as the payload size increases. On the other hand, experimental results reveal an

important observation: although the Raspberry Pi 4 achieves significantly higher absolute throughput due to its 1.5 GHz processor, its micro-level efficiency is lower than that of the ESP32-C6 at a payload size of 128 bytes (170.41 cycles/byte compared to 103.86 cycles/byte). This discrepancy can be attributed to differences in execution environments. Performance measurements on the Raspberry Pi 4, which operates on a multitasking Linux-based system, are affected by context switching overhead and background processes. In contrast, the ESP32-C6 executes the encryption process in a more deterministic manner on a real-time operating system (RTOS), allowing tighter control over CPU cycles and resulting in more efficient execution at the micro-architectural level.

TABLE I. PERFORMANCE ANALYSIS OF ASCON-AEAD128 ON VARIOUS HARDWARE PLATFORMS

Platform	Data size(Bytes)	Operation	Throughput (MB/s)	Cycles/Byte
ESP32-C6	12	Encryption	0.479	334.22
		Decryption	0.463	345.26
	16	Encryption	0.504	317.23
		Decryption	0.493	324.83
	128	Encryption	1.541	103.86
		Decryption	1.527	104.80
Raspberry Pi 4	12	Encryption	2.107	712.53
		Decryption	2.029	739.26
	16	Encryption	2.449	612.48
		Decryption	2.323	645.86
	128	Encryption	8.834	170.41
		Decryption	9.116	164.61

C. Relative Performance Normalization.

To provide a normalized comparative perspective across platforms and algorithms, the measured computational cost was normalized using a relative performance factor. The normalization was performed with respect to ChaCha20-Poly1305, which is widely used as a conventional authenticated encryption scheme for Internet applications. The relative performance factor is defined as:

$$R = \frac{C_{\text{reference}}}{C_{\text{Ascon}}}$$

where $C_{\text{reference}}$ denotes the computational cost of the ChaCha20-Poly1305 reference baseline in cycles per byte, and C_{Ascon} denotes the measured computational cost of Ascon-AEAD128 in cycles per byte on the evaluated platform. A value of R greater than 1 indicates that Ascon-AEAD128 requires fewer CPU cycles per byte than the reference baseline, while a value below 1 indicates higher computational cost under the adopted normalization metric.

TABLE II. NORMALIZED RELATIVE PERFORMANCE OF ASCON-AEAD128 ON ESP32-C6

Platform	Bytes	Operation	Ascon-AEAD128 cost (cycles/byte)	Relative factor R
ESP32-C6	12	Encryption	334.22	0.50
		Decryption	345.26	0.48
	16	Encryption	317.23	0.53
		Decryption	324.83	0.51
	128	Encryption	103.86	1.61
		Decryption	104.80	1.59

The results show that Ascon-AEAD128 becomes more efficient as the payload size increases. For small payloads of 12 and 16 bytes, the relative factor remains below 1 because the initialization and finalization overheads account for a large portion of the total execution cost. However, for a 128-byte payload, the relative factor increases to 1.61 for encryption and 1.59 for decryption, indicating better normalized

efficiency than the reference ChaCha20-Poly1305 [4] baseline under the adopted comparison metric.

D. Cross-Platform Comparative Analysis:

Tables III and IV present a cross-platform comparison of the normalized relative performance of Ascon-AEAD128 against different architectures and other lightweight cryptographic (LWC) algorithms. At a payload size of 128 bytes, the implementation on ESP32-C6 achieves a relative performance factor of 1.61 for encryption and 1.59 for decryption, which provides a practical comparison point against reported results on ARM Cortex-M3 and ESP32 Dev Module platforms.

Furthermore, when compared with representative NIST LWC candidates, Ascon-AEAD128 demonstrates a competitive and well-balanced performance profile under the adopted normalized metric. For the 128-byte payload, the proposed ESP32-C6 implementation achieves relative factors of 1.61 for encryption and 1.59 for decryption, which are comparable to or higher than several reported lightweight cryptography implementations. Although some algorithms such as Xoodoo or Schwaemm may show advantages in specific cases, Ascon-AEAD128 provides a favorable balance between computational efficiency, standardization, and suitability for constrained IoT edge devices.

These results indicate that Ascon-AEAD128 is a highly suitable choice for IoT edge nodes, offering an effective balance between computational efficiency and resource constraints.

TABLE III. COMPARATIVE PERFORMANCE ANALYSIS OF ASCON ACROSS VARIOUS 32-BIT MICROCONTROLLER PLATFORMS

Algorithm	Platform	Encrypt 128 bytes	Decrypt 128 bytes	Encrypt 16 bytes	Decrypt 16 bytes
ASCON-AEAD128(*)	ARM Core M3 (84MHZ)	1.54	1.44	1.78	1.68
ASCON-128a	ARM Core M3 (84MHZ)	1.86	1.70	1.80	1.78
ASCON-AEAD128(*)	ESP32 Dev Module(240MHZ)	0.67	0.46	0.86	0.66
ASCON-128a	ESP32 Dev Module(240MHZ)	0.86	0.88	0.92	0.93
ASCON-AEAD128	ESP32C6 (160MHZ)	1.61	1.59	0.53	0.51

An asterisk (*) indicates algorithms that have been accelerated with assembly code

TABLE IV. PERFORMANCE COMPARISON WITH OTHER LIGHTWEIGHT CRYPTOGRAPHY ALGORITHMS

Algorithm	Platform	Encrypt 128 bytes	Decrypt 128 bytes	Encrypt 16 bytes	Decrypt 16 bytes
ASCON-AEAD128	ESP32-C6 (160MHZ)	1.61	1.59	0.53	0.51
Xoodoo(*)	ARM Core M3 (84MHZ)	1.66	1.51	1.73	1.60
TinyJAMBU128-128(*)	ARM Core M3 (84MHZ)	1.541	1.527	0.504	0.493
GIFT-COFB(*)	ARM Core M3 (84MHZ)	0.93	0.95	1.63	1.61
Schwaemm128-128 (*)	ARM Core M3 (84MHZ)	1.60	1.58	2.84	2.39
Xoodoo	ESP32 Dev Module(240MHZ)	0.84	0.86	0.80	0.82
TinyJAMBU-128(*)	ESP32 Dev Module(240MHZ)	0.60	0.64	1.16	1.15
GIFT-COFB(*)	ARM Core M3 (84MHZ)	0.80	0.83	0.90	0.90
Schwaemm128-128 (*)	ESP32 Dev Module(240MHZ)	1.07	1.06	1.68	1.60

An asterisk (*) indicates algorithms that have been accelerated with assembly code.

E. Data Integrity and Transmission Reliability.

To validate the security and reliability of the AEAD mechanism, the system was evaluated under real-world conditions using a LoRa communication infrastructure. Experimental results obtained from log analysis on the ESP32-C6 transmitter node (Fig. 8) and data streams captured at the Raspberry Pi 4 gateway (Fig. 9) demonstrate correct synchronization of the secret key and nonce throughout the communication process.

In the packet-level reliability experiment, the ESP32-C6 node generated weather data frames with an 11-byte plaintext payload, encrypted each frame using Ascon-AEAD128, and transmitted the resulting packets to the Raspberry Pi 4 gateway through the LoRa link. Each transmitted packet followed the format [Header | Nonce | Ciphertext | Tag], where the nonce was used for decryption and the authentication tag was used to verify data integrity and authenticity. At the gateway side, a packet was considered successful only if the authentication tag was verified correctly and the recovered plaintext matched the original weather data payload.

METRIC	RESULT
NUMBER OF TRANSMITTED PACKETS	10,000
PAYLOAD TYPE	WEATHER DATA
PLAINTEXT PAYLOAD SIZE	11 BYTES
NUMBER OF PACKETS RECEIVED AT GATEWAY	10,000
NUMBER OF SUCCESSFULLY AUTHENTICATED PACKETS	10,000
NUMBER OF AUTHENTICATION FAILURES	0
NUMBER OF PAYLOAD RECOVERY ERRORS	0
OBSERVED AUTHENTICATION FAILURE RATE	0%
OBSERVED PAYLOAD RECOVERY ERROR RATE	0%

AS SHOWN IN TABLE V, A TOTAL OF 10,000 ENCRYPTED WEATHER DATA PACKETS WERE TRANSMITTED FROM THE ESP32-C6 EDGE NODE TO THE RASPBERRY PI 4 GATEWAY. ALL RECEIVED PACKETS PASSED THE ASCON-AEAD128 AUTHENTICATION VERIFICATION PROCESS, AND NO PAYLOAD RECOVERY ERROR WAS OBSERVED. THE ENVIRONMENTAL PARAMETERS, INCLUDING TIMESTAMP, TEMPERATURE, HUMIDITY, AND LIGHT INTENSITY, WERE FULLY RECOVERED AT THE RECEIVER WITHOUT DATA LOSS OR FORMAT CORRUPTION. THEREFORE, UNDER THE TESTED LoRA CONFIGURATION, BOTH THE OBSERVED AUTHENTICATION FAILURE RATE AND THE OBSERVED PAYLOAD RECOVERY ERROR RATE WERE 0%.

However, this result should be interpreted as an empirical observation under the specific experimental setup rather than a universal guarantee for all LoRa channel conditions. Since no failure was observed in 10,000 transmitted packets, the rule-of-three approximation gives an upper bound of approximately 0.03% for the packet failure probability at the 95% confidence level. These results confirm the effectiveness of Ascon-AEAD128 in ensuring confidentiality, integrity, and authentication for small weather-data packets in the tested distributed IoT system.

[illegible]

Fig. 8. Execution logs of the secure LoRa transmission firmware implementing Ascon-AEAD128 algorithm on ESP32-C6.

```
[RX RAW] 18BF5535A56680D897378918373D10A448C98A7B934E6419FA5634A1216268D5CFF3BF330898F043726A38EDF364F4636195E5D78789D209C79BAED66CF
[SUCCESS] Payload: Time:22:26:10,T:31,9,H:75,0,L:139

[RX RAW] 584D702DFB83763040716318E071A958D20384E87499571218880D7878D0AC8B9734E2B84353E479AF01B43076451077233766C3810A2EE1E694BFC45
[SUCCESS] Payload: Time:22:26:11,T:31,9,H:75,0,L:139

[RX RAW] 0E2CE35D632B6F7A28308D8C3165491A79FC0D8778138070A7264E585F7528F37373F7E514121584F966846702A08147E1569F9E5B59CE029F35C
[SUCCESS] Payload: Time:22:26:12,T:31,9,H:75,0,L:139

-----

[RX RAW] 72672774FC888033FAF68CAFDE19A583436EAD01B4795E4EE1E1632F6A90CD56346704D00338144A8540D14C91A38CA62A68572CE85F794B11B
[SUCCESS] Payload: Time:22:26:13,T:31,9,H:75,0,L:139

[RX RAW] F3C0B91150FCE4A8654A451C89E70D707E3D1A0AC4A36CE9D4BCE0EAF9E215ACF7325C9737467E8B6C5F6A028916E790D1F2B4E74D06B480A47574
[SUCCESS] Payload: Time:22:26:14,T:31,9,H:75,0,L:139

[RX RAW] DC2F8E06F0208BFDF7468EAC129273846F52C3593D0CF7E8930C4F9456AEE7C588A8406D69538844770683D43491240C91AC3068D62084DCE7C7C02
[SUCCESS] Payload: Time:22:26:15,T:31,9,H:75,0,L:139

[RX RAW] E175A42D3250F6D507D68C27851814064E8E8D0E1831E7A5B044AF5824685380D5A4F16E42C3C86B27A513181CC9CA9CB858244F8C408F6334A5
[SUCCESS] Payload: Time:22:26:16,T:31,9,H:75,0,L:139

[RX RAW] 9A50C7A8A42C0568771951D0A617EBF61621E244C174A32774D0CE8C38BD0A54CB6E4F92113908194EE1D1C18CE1B83C8877CACA56598557E6F583F041EA8076
[SUCCESS] Payload: Time:22:26:17,T:31,9,H:75,0,L:139
```

Fig. 9. Decryption and data recovery logs on the secure LoRa receiver.

V. CONCLUSION

This paper presented the design and implementation of a secure IoT system based on the Ascon-AEAD128 lightweight cryptographic algorithm. The proposed system was successfully deployed on heterogeneous embedded platforms, including ESP32-C6 and Raspberry Pi 4, interconnected via a LoRa communication infrastructure.

Experimental results demonstrate that Ascon-AEAD128 achieves high computational efficiency and stable performance on resource-constrained devices. The system ensures secure data transmission with full support for confidentiality, integrity, and authentication through the AEAD mechanism. In addition, real-world evaluations confirm reliable end-to-end data delivery without loss or corruption under wireless communication conditions.

Despite these achievements, several limitations remain. The current system is constrained by the low data rate and limited bandwidth of LoRa communication. Moreover, the network architecture is limited to a point-to-point configuration, which restricts scalability for larger IoT deployments.

For future work, the system can be extended to support multi-node architectures using LoRaWAN for scalable device management. Further optimizations can be explored at the implementation level, such as assembly-level tuning to improve performance. Additionally, advanced security mechanisms, including resistance to side-channel and electromagnetic interference attacks, can be investigated to enhance system robustness.

REFERENCES

- [1] *Ascon-Based Lightweight Cryptography Standards for Constrained Devices*, NIST Special Publication 800-232, National Institute of Standards and Technology (NIST), Aug. 2025. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/232/final>.
- [2] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," Submission to NIST Lightweight Cryptography, 2019.
- [3] A. Langley, M. Hamburg, and S. Turner, "ChaCha20-Poly1305 for IETF protocols," RFC 8439, 2018.
- [4] R. Weatherley, "Lightweight cryptography primitives: Performance on 32-bit platforms," 2021. [Online]. Available: <https://rweather.github.io/lightweight-crypto/performance.html>.
- [5] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, "Ascon specification," Graz University of Technology. [Online]. Available: <https://ascon.isec.tugraz.at/specification.html>.
- [6] R. Weatherley, "Performance on 32-bit platforms," Lightweight Crypto Library, 2021. [Online]. Available: <https://rweather.github.io/lwc-finalists/performance.html>.
- [7] *ESP32-C6 Technical Reference Manual*, Espressif Systems, 2023.
- [8] *Raspberry Pi 4 Model B Datasheet*, Raspberry Pi Foundation, 2020.